Contents

1 离线和在线安装 **2**
1.1 龙讯超算云 (Mcloud) 用户 . . . 2
1.2 龙讯一体机 (GPU) 用户 . . . 3
1.3 龙讯一体机 (CPU) 用户 . . . 3

2 离线安装 **3**
2.1 下载离线安装包 . . . 4
2.2 解压安装包 . . . 4
2.2.1 GPU 版本 . . . 4
2.2.2 CPU 版本 . . . 4
2.3 检查编译器版本 . . . 5
2.3.1 GPU 版本 . . . 5
2.3.2 CPU 版本 . . . 5
2.4 执行安装命令 . . . 6
2.5 加载使用 . . . 6
2.6 常见安装错误 . . . 7

3 在线安装 **7**
3.1 MatPL 编译安装 . . . 7
3.1.1 下载源码 . . . 7
3.2 创建 conda 虚拟环境 . . . 8
3.2.1 安装 Anaconda3（已安装请跳过） . . . 8
3.2.2 创建 Python 虚拟环境 . . . 8
3.2.3 安装依赖包 . . . 8
3.2.4 编译安装 . . . 9
3.2.5 编译代码 . . . 10
3.2.6 加载使用 . . . 11
3.3 lammps-MatPL (libtorch) 编译安装 . . . 11
3.3.1 下载 lammps-MatPL (libtorch 版本) 源码 . . . 11
3.3.2 下载 Lammps 源码 . . . 12
3.3.3 复制 lammps-MatPL 接口源码到 Lammps 源码下 . . . 13
3.3.4 加载编译环境 . . . 13
3.3.5 编译 lammps 代码 . . . 13
3.3.6 lammps 加载使用 . . . 14
3.4 Lammps-MatPL (fortran) 编译安装 . . . 15
3.4.1 下载 lammps-MatPL (fortran 版本) 源码 . . . 15
3.4.2 下载 Lammps 源码 . . . 16
3.4.3 复制 Lammps-MatPL (fortran) 源码到 Lammps 源码下 . . . 16
3.4.4 加载编译环境 . . . 16

3.4.5 编译 lammps 代码 16
3.4.6 lammps 加载使用 17
3.5 常见安装错误 17

4 常见安装错误 18
4.1 关于编译环境 18
4.2 找不到 cuda_runtime.h 头文件 18
4.3 NeighConst.so 编译错误 18
4.3.1 错误描述 18
4.3.2 错误原因 19

5 常见运行错误 20
5.0.1 环境变量检查 20
5.0.2 动态库加载错误-mkl 库 20
5.1 Lammps 接口常见运行时错误 21
5.1.1 环境变量检查 21
5.1.2 Lammps NEP 模型 21

1 离线和在线安装

MatPL 提供了 离线安装 和 在线安装 两种方式。对于 龙讯超算云(Mcloud) 用户、龙讯一体机用户，我们已经做了预装，只需要加载即可使用，加载方式如下所示。

1.1 龙讯超算云 (Mcloud) 用户

```
## 加载 MatPL
source /share/app/MATPL/MatPL-2025.3/env.sh

# 或者采用以下方式分步加载
# step1. 加载 python 运行环境
source /share/app/anaconda3/etc/profile.d/conda.sh
module load conda/3-2020.07
conda deactivate
conda activate matpl-2025.3
# step2. 加载 MatPL
module load matpl/2025.3

# 加载 lammps
module load lammps4matpl/2025.3
# 对于 Linear 和 NN 模型的 lammps 接口，我们提供了 cpu 版本的接口，使用 fortran 实现，
↪ 请加载
```

```
module load lammps4matpl/fortran
```

1.2 龙讯一体机 (GPU) 用户

```
# 加载 MatPL 和 lammps
module load intel/2020 cuda/11.8
source /share/app/MATPL/MatPL-2025.3/matpl-env.sh

# 对于 Linear 和 NN 模型的 lammps 接口，我们提供了 cpu 版本的接口，使用 fortran 实现，
↪ 请加载
module load intel/2020
source /share/app/MATPL/MatPL-2025.3/matpl-fortran-env.sh
```

1.3 龙讯一体机 (CPU) 用户

```
# 加载 MatPL 和 lammps
module load intel/2020
source /share/app/MATPL/MatPL-2025.3/matpl-env.sh

# 对于 Linear 和 NN 模型的 lammps 接口，我们提供了 cpu 版本的接口，使用 fortran 实现，
↪ 请加载
module load intel/2020
source /share/app/MATPL/MatPL-2025.3/matpl-fortran-env.sh
```

lammps 接口已经预装了下列功能

- KSPACE
- MANYBODY
- REAXFF
- MOLECULE
- QEQ
- REPLICA
- RIGID
- MEAM
- MC
- SHOCK
- MatPL

2 离线安装

对于 龙讯超算云(`Mcloud`) 用户，已做预装，加载即用 - 对于 GPU 版本，MatPL 离线安装包集成了 MatPL、Lammps 接口源码和依赖的 Python 环境。需要待安装机器提供 `gcc` 编译器、`intel` 编译器套

件（包括 ifort、icc 编译器、mkl 和 mpi 库），以及 nvidia GPU 硬件支持。- 对于 CPU 版本，MatPL 需要待安装机器提供 gcc 编译器、intel 编译器套件（包括 ifort、icc 编译器、mkl 和 mpi 库）即可。

2.1 下载离线安装包

方法一（推荐）邮件获取，建议您发送邮件到 matpl@pwmat、wuxingxing@pwmat.com 或 support@pwmat.com 获取离线安装包。相比于百度网盘，通过邮件链接下载的速度更快（几十倍以上）。

方法二 请访问百度网盘下载，链接如果失效请邮件联系 matpl@pwmat、wuxingxing@pwmat.com 或 support@pwmat.com：⊠ 离线安装包下载 MatPL-2025.3.sh.tar.gz，如果安装 CPU 版本，请选择 CPU 版本下载。

2.2 解压安装包

离线安装包内容较大，我们拆分成了多个小文件，解压安装方式如下所示：

2.2.1 GPU 版本

```
# 合并 4 个文件到 1 个压缩文件
cat matpl-2025.3.sh.tar.gz.part_aa  matpl-2025.3.sh.tar.gz.part_ab
 ↪  matpl-2025.3.sh.tar.gz.part_ac  matpl-2025.3.sh.tar.gz.part_ad
 ↪  matpl-2025.3.sh.tar.gz.part_ae > MatPL-2025.3.sh.tar.gz
# 解压文件
tar -xzvf MatPL-2025.3.sh.tar.gz
```

解压后得到如下文件：MatPL-2025.3.sh，check_offenv.sh

2.2.2 CPU 版本

```
# 合并文件到 1 个压缩文件
cat matpl_cpu-2025.3.sh.tar.gz.part_aa
 ↪  matpl_cpu-2025.3.sh.tar.gz.part_ab
 ↪  matpl_cpu-2025.3.sh.tar.gz.part_ac
 ↪  matpl_cpu-2025.3.sh.tar.gz.part_ad
 ↪  matpl_cpu-2025.3.sh.tar.gz.part_ae  >> MatPL_cpu-2025.3.sh.tar.gz

# 解压文件
tar -xzvf MatPL_cpu-2025.3.sh.tar.gz
```

解压后得到如下文件：MatPL_cpu-2025.3.sh，check_offenv_cpu.sh

2.3 检查编译器版本

2.3.1 GPU 版本

我们推荐使用 `intel2020` 版本，`cuda/11.8`，`gcc 版本 8.n`。这是因为，MatPL 中使用的 `pytorch` 版本为 `2.0` 以上，必须使用 `cuda/11.8` 或更高版本。对于 `intel/2020` 编译套件，使用了它的 `ifort` 和 `icc` 编译器 (`19.1.3`)、`mpi(2019)`、`mkl 库 (2020)`，如果单独加载，请确保版本不低于它们。

大部分的安装失败问题都源于编译器的版本不匹配，我们提供了检查编译器版本的脚本 `check_offenv.sh` 供用户检查环境，执行如下命令

```
sh check_offenv.sh
```

命令执行后会列出需要的编译器版本以及当前检测到的版本，如下是一个正确的环境配置检查结果：

```
ifort version is no less than 19.1, current version is 19.1.
MKL library is installed.
GCC version is exactly 8, current version is 8.
CUDA version is 11.8 or higher, current version is 11.8.89.
nvcc command exists.
```

第 1 行输出了 ifort 编译器要求的版本不低于 19.1，检测到当前的版本是 19.1，满足要求；

第 2 行查找 MKL 库是否存在，检测到已安装，满足要求；

第 3 行输出了 GCC 要求的版本 8.n，检测到当前的 GCC 版本是 8，满足要求;

第 4 行检查 CUDA 版本是否不低于 11.8，检测到当前的版本是 11.8.89，满足要求；

第 5 行检查 nvcc 编译器是否存在，检测到存在，满足要求。

2.3.2 CPU 版本

CPU 离线安装包解压后会得到 `check_offenv_cpu.sh`，执行如下命令

```
sh check_offenv_cpu.sh
```

命令执行后会列出需要的编译器版本以及当前检测到的版本，如下是一个正确的环境配置检查后的结果：

```
ifort version is no less than 19.1, current version is 19.1.
MKL library is installed.
GCC version is exactly 8, current version is 8.
```

第 1 行输出了 ifort 编译器要求的版本不低于 19.1，检测到当前的版本是 19.1，满足要求；

第 2 行查找 MKL 库是否存在，检测到已安装，满足要求；

第 3 行输出了 GCC 要求的版本 8.n，检测到当前的 GCC 版本是 8，满足要求。

CPU 版本不需要 CUDA 和 nvcc 编译器支持。

2.4 执行安装命令

环境检查完毕后，执行如下命令即可完成安装

```
sh MatPL-2025.3.sh [-jN] [-nN]
# -jN 这里 N 为并行编译的核数，例如 sh MatPL-2025.3.sh -j4 将采用 4 核编译。默认采用单
  ↪ 核编译，即 sh MatPL-2025.3.sh
# -nN 这里 N 为 NEP 力场支持的元素类型数量，默认为 20，即支持最多 20 种元素的力场训练，对于通
  ↪ 用力场，可设置为为-n100，例如 sh MatPL-2025.3.sh -j4 -n100 注意，较大的 N 会导致
  ↪ 力场训练速度变慢
```

对于 CPU 版本，执行如下命令：

```
sh MatPL_cpu-2025.3.sh [-jN] [-nN]
# 这里 N 为并行编译的核数，例如 sh MatPL_cpu-2025.3.sh -j4 将采用 4 核编译。默认采用单
  ↪ 核编译，即 sh MatPL_cpu-2025.3.sh
# -nN 这里 N 为 NEP 力场支持的元素类型数量，默认为 20，即支持最多 20 种元素的力场训练，对于通
  ↪ 用力场，可设置为为-n100，例如 sh MatPL_cpu-2025.3.sh -j4 -n100 注意，较大的 N
  ↪ 会导致力场训练速度变慢
```

是否安装成功检查：编译完成后，MatPL_cpu-2025.3 目录下生成如下目录结构：

```
MatPL-2025.3 (或 MatPL_cpu-2025.3)
    ├── lammps-2025.3/
    ├── lammps-fortran/
    ├── matpl-2025.3/  (或 matpl_cpu-2025.3)
    ├── MatPL-2025.3/  (或 MatPL_cpu-2025.3)
    ├── matpl-env.sh
    └── matpl-fortran-env.sh
```

- MatPL-2025.3 为机器学习力场训练平台
- matpl-2025.3 为 python 环境
- lammps-2025.3 为支持 DP 和 NEP 力场的 lammps 力场接口
- lammps-fortran 为支持 NN 和 Linear 力场的 lammps 力场接口
- matpl-env.sh 为 MatPL-2025.3 和 lammps-2025.3 的所有环境变量
- matpl-fortran-env.sh 为 MatPL-2025.3 和 lammps-fortran 的所有环境变量

2.5 加载使用

MatPL 在使用时需要先加载编译时使用的 intel/2020 和 CUDA （对于 GPU 版本）。之后执行如下命令 即可完成 python 环境、MatPL-2025.3 和 lammps 的加载。

```
source /the/path/of/MatPL-2025.3/matpl-env.sh
# cpu 版本 source /the/path/of/MatPL_cpu-2025.3/matpl-env.sh
# 对于 fortran 版本的 lammps 接口:
source /the/path/of/MatPL-2025.3/matpl-fortran-env.sh
```

也可分步骤加载:

```
# step1. 加载 python 环境
source /the/path/MatPL-2025.3/matpl-2025.3/bin/activate
# step2. 加载 MatPL-2025.3
source /the/path/MatPL-2025.3/MatPL-2025.3/env.sh
# step3. 加载 lammps
source /the/path/MatPL-2025.3/lammps-2025.3/env.sh
```

2.6 常见安装错误

MatPL 相关软件的常见 安装错误 和 运行时错误

3 在线安装

对于 龙讯超算云(`Mcloud`) 用户，已做预装，加载即用

在线安装需要您分别编译安装 MatPL 和 lammps 接口。在线安装依赖 `intel` 编译器、`CUDA-11.8` 工具包、`gcc` 编译器以及 `Python` 环境。对于 `intel/2020` 编译套件，使用了它的 `ifort` 和 `icc` 编译器 (`19.1.3`)、`mpi(2019)`、`mkl` 库 (`2020`)，如果单独加载，请确保版本不低于它们。对于 Python 环境，要求 python >= 3.11，torch >= 2.2.0+cu118。

为了编译和运行 MatPL-2025.3，您需要下载源码、安装 conda 环境，并在 conda 环境中安装 MatPL-2025.3 依赖的 Python 环境，之后编译源码。

`MatPL` 和 `lammps-MatPL` 提供了纯 `CPU` 版本。源码相同，GPU 版本除了对 CPU 环境的依赖外，还需要相应的 GPU 环境（CUDA)。

3.1 MatPL 编译安装

3.1.1 下载源码

我们提供了在线拉取代码和下载离线包两种方式编译。

- 通过 github 或 gitee 在线拉取 MatPL 仓库代码

```
git clone https://github.com/LonxunQuantum/MatPL.git MatPL-2025.3
```
或
```
git clone https://gitee.com/pfsuo/MatPL.git MatPL-2025.3
```

- 或下载 release 离线源码包，您可以直接浏览器输入下面的地址下载，或者加前缀 wget 下载:

```
wget
↪ https://github.com/LonxunQuantum/MatPL/archive/refs/tags/MatPL-
↪ 2025.3.zip
或
wget https://gitee.com/pfsuo/MatPL/repository/archive/2025.3
```

下载 release 离线源码包后，通过 unzip 命令解压。

```
# 解压后您将得到一个名称为 MatPL-2025.3 的源码目录
unzip 2025.3.zip
```

3.2 创建 conda 虚拟环境

3.2.1 安装 Anaconda3（已安装请跳过）

这里要求您已经装了 Anaconda3，然后创建一个新 python 虚拟环境（搜索引擎搜索 Linux 安装 anaconda3 教程）。

您可以使用该命令直接下载 Anaconda3 到服务器目录中：

```
curl https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/Anaconda3-
↪ 2023.07-1-Linux-x86_64.sh -o Anaconda3-2023.07-1-Linux-x86_64.sh
# 如果下载失败，请在浏览器输入下面的下载地址，下载后上传的服务器
# https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/Anaconda3-
↪ 2023.07-1-Linux-x86_64.sh
# 您也可以访问网页下载更多版本
↪ https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/
```

3.2.2 创建 Python 虚拟环境

conda 安装完成后，创建虚拟环境，环境中需指定安装 `python3.11` 解释器，其他版本可能会出现依赖冲突或语法不支持等问题，之后的编译工作均在该虚拟环境中进行

```
conda create -n matpl-2025.3 python=3.11 setuptools=68.0.0
# python 版本我们建议 3.11，更高级别的 python 可以在编译时存在一些错误，我们还未兼容。这里需要
↪ 指定 setuptools 版本低于 75.0.0，避免 numpy 和 fortran 做数据格式转换时的错误。
```

虚拟环境安装完成后激活该环境

```
conda activate matpl-2025.3
```

3.2.3 安装依赖包

接下来安装 MATPL 所需的第三方依赖包，我们已经把所有依赖的第三方包写在 requirement.txt 中，只需要在该文件所在目录下执行 pip 命令即可完成所有的依赖包安装。操作过程如下。该步骤会安装 pytorch 等 python 环境较耗时，请您耐心等待。

- matpl-2025.3/requirement.txt 是 MatPL GPU 版本的 python 环境
- matpl-2025.3/requirements_cpu.txt 是 MatPL CPU 版本的 python 环境

```
# 第一步 激活 conda 环境
conda activate matpl-2025.3
# 第二步 进入源码根目录
# 在线下载的源码进入 MatPL-2025.3 目录
cd MatPL-2025.3
# 对于 GPU 版本，请执行
pip install -r requirements.txt
# 对于 CPU 版本，请执行
pip install -r requirements_cpu.txt
```

3.2.4 编译安装

3.2.4.1 检查编译环境 (GPU 版本) 进入 requirement.txt 的同级目录 `src` 目录下。对于 GPU 版本，首先检查 `cuda/11.8`，`intel/2020`，`gcc8.n` 是否加载；检查 `conda` 虚拟环境是否加载。对于 `intel/2020` 编译套件，使用了它的 `ifort` 和 `icc` 编译器 (19.1.3)、`mpi(2019)`、`mkl` 库 (2020)，如果单独加载，请确保版本不低于它们。由于大部分的编译不成功是由编译器版本问题造成的，我们提供了编译环境检测的脚本 `check_env.sh`，位于 `'/src/check/check_env.sh'` 您可以执行该脚本来检查编译环境已经完成准备。

```
cd src
sh ./check/check_env.sh
```

执行后，将输出您的编译环境信息，一个正确的环境如下所示。

```
ifort version is no less than 19.1, current version is 19.1.
MKL library is installed.
GCC version is exactly 8, current version is 8.
PyTorch is installed.
PyTorch version is 2.0 or above, current version is 2.2.
PyTorch is compiled with CUDA 11.8.
CUDA version is 11.8 or higher, current version is 11.8.89.
nvcc command exists.
```

第 1 行输出了 ifort 编译器要求的版本不低于 19.1，检测到当前的版本是 19.1，满足要求；

第 2 行查找 MKL 库是否存在，检测到已安装，满足要求；

第 3 行输出了 GCC 要求的版本 8.n，检测到当前的 GCC 版本是 8，满足要求;

第 4 行检查 python 环境是否已经安装 pytorch，检测到已安装，满足要求；

第 5 行检查 pytroch 版本是否为 2.0 以上，检测到当前版本是 2.2，满足要求；

第 6 行检查 pytroch 版本是否包含 CUDA 支持，检测到包含，满足要求；

第 7 行检查 CUDA 版本是否不低于 11.8，检测到当前的版本是 11.8.89，满足要求；

第 8 行检查 nvcc 编译器是否存在，检测到存在，满足要求。

3.2.4.2 检查编译环境（CPU 版本） 对于 CPU 版本，不需要 CUDA 支持，检测脚本为 check_env_cpu.sh，位于 '/src/check/check_env_cpu.sh'。命令执行后会列出需要的编译器版本以及当前检测到的版本，如下是一个正确的环境配置检查后的结果：

```
ifort version is no less than 19.1, current version is 19.1.
MKL library is installed.
GCC version is exactly 8, current version is 8.
```

第 1 行输出了 ifort 编译器要求的版本不低于 19.1，检测到当前的版本是 19.1，满足要求；

第 2 行查找 MKL 库是否存在，检测到已安装，满足要求；

第 3 行输出了 GCC 要求的版本 8.n，检测到当前的 GCC 版本是 8，满足要求。

3.2.5 编译代码

如果您的环境满足上述检测，接下来进行代码编译。执行如下命令开始编译：

```
sh clean.sh
sh build.sh [-jN] [-nN]
# -jN 这里 N 为并行编译的核数，例如 sh build.sh -j4 将采用 4 核编译。默认采用单核编译，即
 ↪ sh build.sh
# -nN 这里 N 为 NEP 力场支持的元素类型数量，默认为 20，即支持最多 20 种元素的力场训练，对于通
 ↪ 用力场，可设置为为-n100，例如 sh build.sh -j4 -n100。注意，较大的 N 会导致力场训练速
 ↪ 度变慢
```

如果您在编译过程中出错，请在MATPL 常见安装错误总结 中查询。

如果仍未解决您的问题，请将您的机器环境信息、编译错误日志以及您执行的编译操作过程描述 发送到邮箱 matpl@pwmat、wuxingxing@pwmat.com 或 support@pwmat.com，我们将及时联系您处理。

编译完成后，最后输出如下信息：

```
==================================
MatPL has been successfully installed. Please load the MatPL environment var
You can load the environment variables by running (recommended):

  source /the/path/of/MatPL-2025.3/env.sh

Or by executing the following commands:
```

```
export PYTHONPATH=/the/path/of/MatPL-2025.3:$PYTHONPATH
export PATH=/the/path/of/MatPL-2025.3/src/bin:$PATH
```

==========

编译完成后，将在代码的根目录下生成一个 env.sh 文件，包含 MatPL 的环境变量，执行以下命令即可完成加载

```
source /the/path/of/MatPL-2025.3/env.sh
```

也可以通过如下命令加载

```
export PYTHONPATH=/the/path/of/MatPL-2025.3:$PYTHONPATH
export PATH=/the/path/of/MatPL-2025.3/src/bin:$PATH
```

3.2.6 加载使用

使用 MatPL 前需要加载它的依赖环境，加载 python 环境、cuda、MatPL 环境变量（CPU 版本不需要加载 CUDA)。

```
conda activate matpl-2025.3
module load cuda/11.8-share
source /the/path/of/MatPL-2025.3/env.sh
```

之后即可使用 MatPL 命令开始训练，使用教程请参考 教学案例

3.3 lammps-MatPL (libtorch) 编译安装

lammps-MatPL (libtorch 版本) CPU 与 GPU 版本源码相同，区别只在编译上。

lammps-MatPL (libtorch 版本) 用于 MatPL 的 DP 和 NEP 力场。对于 NN 和 Linear 力场，提供了 fortran 版本的 接口，安装请参考 lammps-fortran 编译安装。

lammps-MatPL (libtorch 版本) 由于用到了 libtorch 加速，因此 编译 Lammps 接口前，需要您已经成功编译安装 MatPL。安装过程中，需要您下载 lammps 源码、加载编译器、编译源码，过程如下所示。

3.3.1 下载 lammps-MatPL (libtorch 版本) 源码

lammps-MatPL (libtorch 版本) 力场接口源码位于 MatPL 源码目录 `lmps/lammps-libtorch` 下，您也可以通过 github 下载 下载 lammps-MatPL (libtorch 版本) 源码，或下载 release 包。- 通过 github 或 gitee clone 源码:

```
git clone https://github.com/LonxunQuantum/lammps-MatPL.git
或
git clone https://gitee.com/pfsuo/lammps-MatPL.git
```

- 或下载 release 包:

```
wget https://github.com/LonxunQuantum/lammps-
↪ MatPL/archive/refs/tags/2025.3.zip
或
wget https://gitee.com/pfsuo/lammps-MatPL/repository/archive/2025.3

unzip 2025.3.zip     # 解压源码
```

MatPL 力场接口源码目录如下所示

```
├── README.md
├── LICENSE
├── Makefile.mpi
├── MATPL/
│   ├── dftd3para.h
│   ├── nep_cpu.cpp
│   ├── nep_cpu.h
│   ├── NEP_GPU
│   ├── pair_matpl.cpp
│   └── pair_matpl.h
├── dp_lmps_demo/
│   ├── in.lammps
│   ├── jit_dp.pt
│   ├── lmp.config
│   ├── runcpu.job
│   └── rungpu.job
└── nep_lmps_demo/
    ├── nep_lmps/
    └── nep_lmps_deviation/
```

MATPL 为力场接口源码，Makefile.mpi 为编译配置文件，dp_lmps_demo 和 nep_lmps_demo 分别是 DP 和 NEP 的力场 MD 测试例子。

3.3.2 下载 Lammps 源码

lammps 源码请访问 lammps github 仓库 下载，这里推荐下载 `stable` 版本。

3.3.3 复制 lammps-MatPL 接口源码到 Lammps 源码下

- 复制 Makefile.mpi 文件到 lammps/src/MAKE/目录下
- 复制 MATPL 目录到 lammps/src/目录下

3.3.4 加载编译环境

lammps-MatPL (libtorch 版本) 的 lammps 接口编译，依赖的编译器环境与 MatPL 相同。如果安装 CPU 版本，则不用加载 CUDA。

```
# 加载编译器
module load cuda/11.8-share intel/2020
# 此为 gcc 编译器，您可以加载自己的 8.n 版本
source /opt/rh/devtoolset-8/enable
# 加载 python 环境
source /the/path/anaconda3/etc/profile.d/conda.sh
conda activate matpl-2025.3
```

3.3.5 编译 lammps 代码

3.3.5.1 为了使用 NEP 模型的 GPU 版本，需要您先将 NEP 的 c++ cuda 代码编译为共享库文件 如果安装 CPU 版本，则跳过该步即可。

```
cd lammps/src/MATPL/NEP_GPU
make clean
make [-jN]
# 这里 N 为并行编译的核数，例如 make -j4 将采用 4 核编译。默认采用单核编译，即 make
# 编译完成后您将得到一个/lammps/src/libnep_gpu.so 的共享库文件
```

3.3.5.2 编译 lammps 接口

```
cd lammps/src
make yes-MATPL
# 以下 lammps 中常用软件，推荐在安装时顺带安装
make yes-KSPACE
make yes-MANYBODY
make yes-REAXFF
make yes-MOLECULE
make yes-QEQ
make yes-REPLICA
make yes-RIGID
make yes-MEAM
make yes-MC
```

```
make yes-SHOCK
# 开始编译
make clean-all
make mpi [-jN] [mode=shared]
# 这里 N 为并行编译的核数，例如 make mpi -j4 mode=shared 将采用 4 核编译。默认采用单核编
 ↪ 译，即 make mpi mode=shared
# mode=shared 可选项，表示同时编译 lammps 共享库，该共享库可用于各类 python 接口中，推荐安
 ↪ 装
```

如果编译过程中找不到 `cuda_runtime.h` 头文件，请在 `src/MAKE/Makefile.mpi` 文件的第 24 行替换为您自己的 CUDA 路径，`/the/path/cuda/cuda-11.8`，`cuda_runtime.h` 位于该目录下的 `include` 目录下。CPU 版本不存在该问题。

```
CUDA_HOME = $(CUDADIR)
替换为 CUDA_HOME = /the/path/cuda/cuda-11.8
```

编译完成将在窗口输出如下信息，并在 lammps 源码根目录生成一个 env.sh 文件，使用 lammps 前加载该文件即可。

```
mpicxx -g -O3 -std=c++17 -L/the/path/of/lammps/src/Obj_shared_mpi/..
 ↪ -lnep_gpu -L/share/app/cuda/cuda-11.8/lib64 -lcudart
 ↪ -L/share/app/cuda/cuda-11.8/lib64 -lcudart main.o      -L.
 ↪ -llammps_mpi      -ldl  -o ../lmp_mpi
size ../lmp_mpi
   text    data     bss     dec     hex filename
   5042    1032       8    6082    17c2 ../lmp_mpi
============================
LAMMPS has been successfully compiled. Please load the LAMMPS
 ↪ environment variables before use.
You can load the environment variables by running (recommended):

    source /the/path/of/lammps/env.sh

Or by executing the following commands:
    export PATH=/the/path/of/lammps/src:$PATH
    export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/the/path/of/lammps/src
============================
make[1]: Leaving directory `/the/path/of/lammps/src/Obj_shared_mpi'
```

3.3.6 lammps 加载使用

使用 lammps 需要加载它的依赖环境，加载 intel(mpi)、cuda、lammps 环境变量。CPU 版本不需要加载 CUDA。

```
module load intel/2020 cuda/11.8-share
source /the/path/of/lammps/env.sh
```

之后即可使用如下命令启动 lammps 模拟

```
mpirun -np 4 lmp_mpi -in in.lammps
```

更详细的使用，请参考 - MatPL 操作演示：NEP lammps - MatPL 操作演示：DP lammps

3.4 Lammps-MatPL (fortran) 编译安装

lammps-MatPL (fortran 版本) 用于 MatPL 的 NN 和 Linear 力场，未提供 GPU 加速。安装过程中，需要您下载 lammps 源码、加载编译器、编译源码，过程如下所示。

3.4.1 下载 lammps-MatPL (fortran 版本) 源码

lammps-MatPL (fortran 版本) 力场接口源码位于 MatPL 源码目录 `lmps/lammps-libtorch` 下，您也可以通过 github 下载 下载 lammps-MatPL (fortran 版本) 源码，或下载 release 包。- 通过 github 或 gitee clone 源码:

```
git clone -b fortran
 ↪  https://github.com/LonxunQuantum/lammps-MatPL.git
```

或

```
git clone -b fortran https://gitee.com/pfsuo/lammps-MatPL.git
```

- 或下载 release 包:

```
wget https://github.com/LonxunQuantum/lammps-
 ↪  MatPL/archive/refs/tags/matpl-fortran-2025.3.zip
```

或

```
wget https://gitee.com/pfsuo/lammps-MatPL/repository/archive/2025.3

unzip 2025.3.zip     # 解压源码
```

MatPL 力场接口源码目录如下所示

```
├── lmps-examples/
│   ├── linear_lmps/
│   └── nn_lmps/
├── MATPL/
├── Makefile.mpi
└── README.md
```

README.md 是安装说明，MATPL 为力场接口源码，Makefile.mpi 为编译配置文件，nn_lmps 和 linear_lmps 分别是 NN 和 Linear 的力场 MD 测试例子。

3.4.2 下载 Lammps 源码

lammps 源码请访问 lammps github 仓库 下载，这里推荐下载 `stable` 版本。

3.4.3 复制 Lammps-MatPL (fortran) 源码到 Lammps 源码下

- 复制 Makefile.mpi 文件到 lammps/src/MAKE/目录下
- 复制 MATPL 目录到 lammps/src/目录下

3.4.4 加载编译环境

首先检查 `intel/2020`，`gcc8.n` 是否加载；

- 对于 `intel/2020` 编译套件，使用了它的 `ifort` 和 `icc` 编译器 (`19.1.3`)、`mpi(2019)`、`mkl` 库 (`2020`)，如果单独加载，请确保版本不低于它们。

```
# 加载编译器
module load intel/2020
# 此为 gcc 编译器，您可以加载自己的 8.n 版本
source /opt/rh/devtoolset-8/enable
```

3.4.5 编译 lammps 代码

3.4.5.1 编译 fortran 力场库文件

```
cd lammps/src/MATPL/fortran_code
make clean
make [-jN]
# 编译完成后您将得到一个/lammps/src/MATPL/f2c_calc_energy_force.a 文件
# 这里 N 为并行编译的核数，例如 make -j4 将采用 4 核编译。默认采用单核编译，即 make
```

3.4.5.2 编译 lammps 接口

```
cd lammps/src
make yes-MATPL
# 以下 lammps 中常用软件，推荐在安装时顺带安装
make yes-KSPACE
make yes-MANYBODY
make yes-REAXFF
make yes-MOLECULE
make yes-QEQ
make yes-REPLICA
make yes-RIGID
```

```
make yes-MEAM
make yes-MC
make yes-SHOCK
# 开始编译
make clean-all
make mpi [-j4] [mode=shared] # 这里 4 为并行编译数量，shared 为编译出一个共享库文件，
 ↪ 可以用于 python 相关操作中
# 这里 N 为并行编译的核数，例如 make mpi -j4 mode=shared 将采用 4 核编译。默认采用单核编
 ↪ 译，即 make mpi mode=shared
# mode=shared 可选项，表示同时编译 lammps 共享库，该共享库可用于各类 python 接口中，推荐安
 ↪ 装
```

编译完成将在窗口输出如下信息，并在 lammps 源码根目录生成一个 env.sh 文件，使用 lammps 前加载该文件即可。

```
============================
LAMMPS has been successfully compiled. Please load the LAMMPS
 ↪ environment variables before use.
You can load the environment variables by running (recommended):

    source the/path/of/lammps/env.sh

Or by executing the following commands:
    export PATH=the/path/of/lammps/src:$PATH
    export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:the/path/of/lammps/src
    export LAMMPS_POTENTIALS=the/path/of/lammps/potentials
============================
make[1]: Leaving directory `the/path/of/lammps/src/Obj_shared_mpi'
```

3.4.6 lammps 加载使用

使用 lammps 需要加载它的依赖环境，加载 intel(mpi)、lammps 环境变量。

```
module load intel/2020
source /the/path/of/lammps/env.sh
```

之后即可使用如下命令启动 lammps 模拟

```
mpirun -np 4 lmp_mpi -in in.lammps
```

更详细的使用，请参考 - MatPL 操作演示：Linear lammps - MatPL 操作演示：NN lammps

3.5 常见安装错误

MatPL 相关软件的常见 安装错误 和 运行时错误

4 常见安装错误

4.1 关于编译环境

大部分的安装失败问题，都来源于编译安装环境版本不匹配，或找不到相关环境变量。请先检查下列编译器是否已正确安装，并且版本适配。

我们推荐使用 `intel2020` 版本，`cuda/11.8`，`cmake 版本 >= 3.21`，`gcc 版本 8.n`。MatPL 中使用的 `pytorch` 版本为 `2.0` 以上，必须使用 `cuda/11.8` 或更高版本。

对于 `intel/2020` 编译套件，使用了它的 `ifort` 和 `icc` 编译器 (`19.1.3`)、`mpi(2019)`、`mkl 库 (2020)`，如果单独加载，请确保版本不低于它们。

您可以通过位于源码根目录的 src/check/check_env.sh 脚本检查环境。一个正确的环境如下所示。

```
1. CUDA version is 11.8.
2. nvcc command exists.
3. ifort version is no less than 19.1, current version is 19.1.
4. MKL library is installed.
5. GCC version is not 8.x, current version is 8.
6. PyTorch is installed.
7. PyTorch version is 2.0 or above, current version is 2.2.
```

4.2 找不到 `cuda_runtime.h` 头文件

如果编译过程中找不到 `cuda_runtime.h` 头文件，请在 `src/MAKE/Makefile.mpi` 文件的 第 24 行 替换为您自己的 CUDA 路径，`/the/path/cuda/cuda-11.8`，`cuda_runtime.h` 位于该目录下的 `include` 目录下。

```
CUDA_HOME = $(CUDADIR)
替换为 CUDA_HOME = /the/path/cuda/cuda-11.8
```

4.3 NeighConst.so 编译错误

4.3.1 错误描述

在编译 fortran 代码过程中出现如下错误

```
ifort -O3 least_squares.f90 counts_atom.f90 scan_title.f90 transform_to_upp
      find_neighbore00.f90 find_neighbore.f90 find_feature_deepMD2.f90 \
         gen_deepMD2_feature.f90 \
    -o gen_deepMD2_feature.x -mkl
```

```
python3 -m numpy.f2py -c -m NeighConst --fcompiler=intelem --
compiler=intelem -L/share/app/intel2020ucompilers_and_libraries_2020.4.304
lmkl_rt NeighConst.f90
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/__in__.py", line 5, in <module>
    main()
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/f22e.py", line 766, in main
    run_compile()
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/f22e.py", line 594, in run_compile
    build_backend = f2py_build_generator(backend_key)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/_bkends/__init__.py", line 6, in f2py_build_generator
    from ._distutils import DistutilsBackend
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/f2py/_bkends/_distutils.py", line 3, in <module>
    from numpy.distutils.core import setup, Extension
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/distuti/core.py", line 24, in <module>
   from numpy.distutils.command import config, config_compiler, \
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/distuti/command/config.py", line 19, in <module>
   from numpy.distutils.mingw32ccompiler import generate_manifest
 File "/data/home/wuxingxing/anaconda3/envs/pwmlff-2024.5/lib/python3.11/
packages/numpy/distuti/mingw32ccompiler.py", line 27, in <module>
   from distutils.msvccompiler import get_build_version as get_build_msvc_v
ModuleNotFoundError: No module named 'distutils.msvccompiler'
make: *** [NeighConst.so] Error 1
make: Leaving directory `/data/home/wuxingxing/codespace/PWMLFF_gpu/src/pr
make: Entering directory `/data/home/wuxingxing/codespace/PWMLFF_gpu/src/p
```

4.3.2 错误原因

该错误出自 setuptools 版本不匹配，一般是版本过高造成的，您需要降低 setuptools，执行如下命令：

```
# 卸载 setuptools
```

```
$ pip uninstall setuptools
# 清除本地缓存
$ pip cache purge
# 重新安装 setuptools
$ pip install setuptools==68.0.0
# 在我们的测试中不高于 68.0.0 即可
```

5 常见运行错误

5.0.1 环境变量检查

由于未正确加载或者未加载相关环境变量，导致的运行时错误，一般表现为找不到 MatPL 命令 或者 一些 ***.so 的动态库缺失。此时请检查下列环境变量是否都已经加载。

```
# python 环境，是否激活了 python 环境
source /the/path/etc/profile.d/conda.sh
conda activate matpl-2025.3

# intel 和 cuda 工具集是否加载
module load intel/2020 cuda/11.8

# MatPL 的环境变量是否加载
source /the/path/MatPL-2025.3/env.sh
```

5.0.2 动态库加载错误-mkl 库

5.0.2.1 错误描述

```
    exec(code, run_globals)
  File "/the/path/MatPL-2025.3/main.py", line 6, in <module>
    from src.user.dp_work import dp_train, dp_test
  File "/the/path/MatPL-2025.3/src/user/dp_work.py", line 6, in
  ↪  <module>
    from src.PWMLFF.dp_network import dp_network
  File "/the/path/MatPL-2025.3/src/PWMLFF/dp_network.py", line 42, in
  ↪  <module>
    import src.pre_data.dp_mlff as dp_mlff
  File "/the/path/MatPL-2025.3/src/pre_data/dp_mlff.py", line 11, in
  ↪  <module>
    from src.lib.NeighConst import neighconst
ImportError: libmkl_rt.so: cannot open shared object file: No such
 ↪  file or directory
```

5.0.2.2 解决方法 没有加载 Intel Math Kernel Library (MKL)，intel/2020 模块（Intel Parallel Studio XE 2020 或 Intel oneAPI Toolkits 2020 版本中的一个模块化软件）通常包含 Intel MKL 库。加载这个模块时，MKL 库将可用于你的编译和运行环境中。

```
module load intel/2020
```

5.1 Lammps 接口常见运行时错误

5.1.1 环境变量检查

由于未正确加载或者未加载相关环境变量，导致的运行时错误，一般表现为找不到 `lmp_mpi` 命令，或者一些 `***.so` 的动态库缺失。此时请检查下列环境变量是否都已经加载。

```
#1. 用于 mpirun 命令
module load intel/2020

#2. 加载 lammps 环境变量
source /the/path/of/lammps/env.sh

#3. 运行 lammps 命令
mpirun -np 4 lmp_mpi -in in.lammps
```

5.1.2 Lammps NEP 模型

5.1.2.1 错误描述 nep 模型在 GPU 接口中运行一段时间后，出现如下错误：

```
    ......
    97000   1293.8659     -70999.672      35.957737     -70963.715
 ↪  13.66254       13.66254       12.50455       2334.1619
    98000   1191.7602     -71009.412      33.120127     -70976.292
 ↪  13.577541      13.577541      12.426755      2290.8676
    99000   1219.1286     -71013.893      33.880718     -70980.012
 ↪  13.488421      13.48842       12.345188      2246.0524
CUDA Error:
    File:       utilities/gpu_vector.cu
    Line:       117
    Error code: 700
    Error text: an illegal memory access was encountered

=====================================================================================
=   BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
=   RANK 0 PID 5490 RUNNING AT gn59
=   KILLED BY SIGNAL: 9 (Killed)
```

```
===================================================================================

===================================================================================
=   BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
=   RANK 1 PID 5491 RUNNING AT gn59
=   KILLED BY SIGNAL: 9 (Killed)
===================================================================================

===================================================================================
=   BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
=   RANK 2 PID 5492 RUNNING AT gn59
=   KILLED BY SIGNAL: 9 (Killed)
===================================================================================
```

5.1.2.2 解决方法 该错误一般是 MD 运行一段时间后，力场拟合精度下降，导致部分原子的邻居列表变化较大，超过了初始设置的最大邻居数目，调大最大邻居数设置即可。调整方法：在 nep 的力场文件，如下所示的 nep_to_lmps.txt：

```
nep4   2 O Si
cutoff 6.0 5.0
n_max  4 4
basis_size 12 12
l_max  4 2 1
......
```

在 `cutoff` 所在行，修改为如下

```
cutoff 6.0 5.0 500 400
```

这里 `500` 是 两体 (radial_cutoff) cutoff 对应的最大邻居数，`400` 是多体（angular_cutoff）项对应的最大邻居数。

请注意，如果出现该错误，请先检查 md 轨迹文件是否正常，是否 md 本身跑蹦了。